

RC Car Project — Progress Report & Replication Guide

Date: 16 July 2026 **Goal:** A 4-wheel differential-drive RC car controlled from a browser, with live sensor telemetry. Raspberry Pi Zero 2 W as the brain, one L298N driving four TT motors (two per channel), controlled over WiFi.

This report describes exactly what we have done so far, in order, so anyone can replicate it on the same setup.

1. The Setup

Hardware

Part	Role
Raspberry Pi Zero 2 W	Runs the server, reads sensors, drives the motor board
L298N dual H-bridge (×1)	Drives the motors. Left motor pair on channel A, right pair on channel B
4× TT gear motors	Two wired in parallel per L298N channel
2× US-100 ultrasonic	Front and rear distance
6× IR proximity (LM393)	Near-obstacle dots
MPU6050	IMU (accelerometer + gyroscope) over I ² C
3S3P 18650 pack (~11.1 V)	Motor power, direct to L298N +12V
LM2596 buck converter	5 V rail for Pi, sensors, and L298N logic

Platforms

- Pi:** Raspberry Pi OS **Trixie** (Debian 13), kernel `6.18`, Python 3.13, user `eams-pi`, IP `172.20.10.2` (phone hotspot network).
- Dev machine:** Linux laptop, files in `~/Downloads/iso/pi/rccar/`, IP `172.20.10.3`.

Key design fact

Two motors are paralleled per L298N channel, so stall current (~2–3 A) exceeds the chip's realistic 2 A rating. **The software is the thermal protection.** All safety limits (duty cap, slew limit, deadman, stall cutoff) live server-side in the motor code.

2. The Codebase (generated first, on the dev machine)

The full project was generated from a written requirements document. File layout:

```
rccar/  
├─ config.py          # ALL pins and tunable constants (BCM numbering)  
├─ motors.py          # motor thread + safety layer (only file touching motor  
pins)  
├─ sensors.py         # sensor thread: ultrasonics, IR, MPU6050  
├─ server.py          # Flask + flask-sock server, entry point, --dry-run flag  
├─ requirements.txt  
├─ rccar.service      # systemd unit (installed later)  
├─ static/            # index.html, app.js, style.css – the browser app  
└─ setup/            # hostapd.conf, dnsmasq.conf – AP mode, unused for now
```

How the software works (overview)

Three threads on the Pi:

1. **Web/socket thread** — serves the static page over HTTP on port 8080 and owns one WebSocket that carries commands down and telemetry up.
2. **Sensor thread** — samples everything on a 20 Hz tick into a shared dictionary (under a lock). Ultrasonics fire *alternately* (front on even ticks, rear on odd) to avoid cross-talk. IMU is read raw over I²C.
3. **Motor thread** — a 50 Hz loop that is the only code allowed to touch motor pins. Every tick it applies, in order:
 - **Deadman:** no command packet for 300 ms → motors to zero.
 - **Stall cutoff:** duty commanded but no gyro motion for 1 s → cut, latch a `stall` flag, require the client to press RESET.
 - **Slew limit:** duty ramps to target over ~200 ms, never steps.
 - **Zero-cross coast:** on direction reversal, force 80 ms of coast first.
 - **Duty cap:** hard clamp at **55 %** PWM, applied last.

GPIO library: `pigpio` via the `pigpiod` daemon — chosen because it uses DMA timing (precise PWM and microsecond echo timestamps, independent of CPU load).

Browser client: plain HTML/JS, no framework. Touch joystick + Gamepad API → arcade mix (`left = y + x`, `right = y - x`, normalized) → JSON commands at 20 Hz, *including zeros when idle* (silence is how the deadman fires). It renders distance bars, six IR dots, scaled IMU values, actual applied duty, and STALL/DEADMAN banners.

`server.py --dry-run` stubs all GPIO so the whole stack can be tested on a laptop with no hardware. This was done and passed before deploying.

3. Problem 1: `pigpio` is gone from apt on Trixie

Planned command:

```
bash
```

```
sudo apt install -y pigpio python3-pigpio python3-flask python3-venv
```

Result:

```
Error: Package 'pigpio' has no installation candidate
```

Why: Raspberry Pi OS Trixie (Debian 13) dropped the `(pigpio)` package.

Fix chosen: build pigpio from source. (Alternative was reflashing Bookworm Legacy.)

3.1 Install build tools

```
bash
```

```
sudo apt install -y python3-venv python3-pip git build-essential
```

Objective: compiler (`(gcc)`, `(make)`), git to fetch source, venv/pip for Python later. All were already present.

3.2 Clone and compile

```
bash
```

```
cd /tmp
git clone https://github.com/joan2937/pigpio
cd pigpio
make
```

Objective: download the pigpio source and compile the C library (`(libpigpio.so)`), the daemon (`(pigpiod)`), and the command tool (`(pigs)`). Compiled clean.

3.3 Install

```
bash
```

```
sudo make install
```

Result: installed the libraries to `(/usr/local/lib)` and binaries (`(pigpiod)`, `(pigs)`) to `(/usr/local/bin)`, **then failed** at the last step:

```
ModuleNotFoundError: No module named 'distutils'
```

Why: the Makefile's final step installs the *Python* pigpio module using `(distutils)`, which was removed in Python 3.12+. **This failure did not matter** — everything the daemon needs was already installed, and we install the Python module from PyPI inside the venv instead (step 5).

3.4 Register the libraries

```
bash

sudo ldconfig
```

Objective: refresh the dynamic-linker cache so the system can find the freshly installed `.so` files in `/usr/local/lib`.

3.5 Create a systemd service for the daemon

The apt package normally ships `pigpiod.service`; building from source doesn't, so we wrote one:

```
bash

sudo tee /etc/systemd/system/pigpiod.service << 'EOF'
[Unit]
Description=pigpio daemon
After=network.target

[Service]
Type=forking
ExecStart=/usr/local/bin/pigpiod
ExecStop=/bin/systemctl kill pigpiod

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now pigpiod
```

Objective: run `pigpiod` at boot with **default flags** (important — no `-t 0`). `daemon-reload` makes systemd re-read unit files; `enable --now` starts it immediately and at every boot.

3.6 Verify

```
bash

systemctl status pigpiod    # → active (running)
pigs t                     # → 3214257916 (a tick count = daemon answering)
```

`pigs t` asks the daemon for its current tick; getting a number back proves the daemon is alive and reachable.

4. Problem 2: transferring the code (two wrong paths)

Attempt 1 — wrong remote user

```
bash
```

```
rsync -avz --delete ./rccar/ eams-pi@172.20.10.2:/home/pi/rccar/  
# mkdir "/home/pi/rccar" failed: No such file or directory
```

Why: the Pi's user is `eams-pi`, not `pi`, so `/home/pi` doesn't exist.

Attempt 2 — wrong local path

```
bash
```

```
rsync -avz --delete ./rccar/ eams-pi@172.20.10.2:/home/eams-pi/rccar/  
# change_dir ".../rccar" failed - no rccar/ in the current directory
```

Why: the shell was not in the folder containing `rccar/`.

Working command

Run from the directory that actually contains `rccar/`:

```
bash
```

```
rsync -avz --delete rccar/ eams-pi@172.20.10.2:/home/eams-pi/rccar/
```

All 14 files transferred. *Flags:* `(-a)` preserve structure/permissions, `(-v)` verbose, `(-z)` compress, `(--delete)` mirror exactly (remove remote files not present locally). The trailing `/` on `rccar/` means "copy the *contents* of this folder".

Follow-up needed: `rccar.service` was written for user `pi`; before installing it, change three lines:

```
ini
```

```
User=eams-pi  
WorkingDirectory=/home/eams-pi/rccar  
ExecStart=/home/eams-pi/rccar/.venv/bin/python server.py
```

5. Python environment on the Pi

```
bash
```

```
cd ~/rccar  
python3 -m venv .venv  
.venv/bin/pip install flask flask-sock pigpio
```

Objective: create an isolated Python environment and install the three dependencies.

Because apt's `python3-pigpio` doesn't exist on Trixie, the pigpio **Python client** comes from PyPI (`pipio 1.78`) — this replaces the step that failed in 3.3. Note: `--system-site-packages` from the original plan is **not needed** anymore.

One transient network hiccup during download (`Retrying...`) — pip retried and succeeded. Installed: flask 3.1.3, flask-sock 0.7.0, pigpio 1.78 + dependencies.

6. Enable I²C and first run

```
bash

sudo raspi-config nonint do_i2c 0
```

Objective: enable the I²C bus for the MPU6050 (non-interactive; `0` means "on").

```
bash

i2cdetect -y 1
# -bash: i2cdetect: command not found
```

Open item: `i2c-tools` isn't installed. Fix: `sudo apt install -y i2c-tools`, then re-run `i2cdetect -y 1` and expect the IMU at address `0x68`.

First run (wheels off the ground)

```
bash

.venv/bin/python server.py
```

Results, in order of appearance:

Log line	Meaning	Status
<code>MPU6050 init failed ('I2C write failed')</code>	The IMU was not reachable over I ² C	⚠ open item, see §7
<code>Running on http://172.20.10.2:8080</code>	Server up; browser at that URL loaded the app (200s for <code>/</code> , <code>app.js</code> , <code>style.css</code>)	✓
<code>client connected</code>	WebSocket handshake worked; telemetry streaming	✓
<code>DEADMAN tripped: no command for 300 ms</code>	Fires whenever the client goes quiet (tab in background, disconnect). This is the safety net working	✓ expected
<code>STALL cut ... awaiting client reset</code> / <code>stall latch cleared by client reset</code>	Because the IMU is dead, the code sees "duty commanded but no motion" after 1 s and cuts. RESET button works	⚠ expected side-effect of the IMU being down
<code>Bad HTTP/0.9 request type ('\x16\x03\x01...')</code> spam	A browser tried https:// against our plain-HTTP server; <code>\x16\x03\x01</code> is a TLS handshake. Harmless — use <code>http://</code> explicitly	✓ ignore
<code>client disconnected – deadman will zero motors</code>	Clean disconnect path works	✓

Bottom line: server, web app, WebSocket, deadman, stall latch, and reset all verified working on real hardware. The multiplication of stall trips is *correct behavior given a dead IMU* — the safety layer refuses to run motors it can't confirm are moving.

7. Current state & next steps

Done:

- ✓ Codebase generated and dry-run tested on laptop
- ✓ pigpio built from source, daemon running as a service, verified with `pigs t`
- ✓ Code transferred to `/home/eams-pi/rccar`
- ✓ venv + dependencies installed
- ✓ I²C enabled
- ✓ Server runs; web app loads; WebSocket, deadman, stall/reset verified

To do:

1. `sudo apt install -y i2c-tools` → `i2cdetect -y 1`. If `0x68` does **not** appear, check MPU6050 wiring (VCC to 3.3 V rail per design, SDA→GPIO2, SCL→GPIO3, common ground). The stall trips will stop once the IMU answers.
2. Edit the 3 user paths in `rccar.service` (§4), then install it:

bash

```
sudo cp ~/rccar/rccar.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now rccar  
journalctl -u rccar -f
```

3. First-run hardware checklist: wheels off ground → confirm L298N 5 V jumper removed → confirm echo-pin voltage dividers → logic rail only, verify telemetry → connect motor rail → verify deadman (close tab, wheels stop < 300 ms) → verify direction mapping.
4. Measure real motor stall current; adjust `DUTY_CAP` in `config.py` if per-motor stall > ~1.5 A.

Glossary

- **BCM numbering** — Naming GPIO pins by the Broadcom chip's numbers (GPIO 12) instead of physical header positions.
- **Deadman (switch)** — Safety rule: if control packets stop arriving, the motors stop automatically.
- **Differential drive / arcade mix** — Steering by running left and right wheels at different speeds; arcade mix converts one joystick (x, y) into left/right values.
- **distutils** — Old Python packaging module, removed in Python 3.12; caused the `make install` failure.
- **Duty cycle / duty cap** — Fraction of time the PWM signal is "on" (= motor power). The cap limits it to 55 % to protect the L298N.
- **H-bridge (L298N)** — Circuit that lets a motor run in both directions; the L298N has two of them.
- **I²C** — Two-wire bus (SDA data, SCL clock) used to talk to the MPU6050.
- **IMU** — Inertial Measurement Unit; combined accelerometer + gyroscope (MPU6050).
- **ldconfig** — Refreshes Linux's cache of shared libraries so newly installed `.so` files are found.
- **pigpio / pigpiod / pigs** — GPIO library, its background daemon, and its command-line tool. Uses DMA for precise timing.
- **PWM** — Pulse-Width Modulation; rapidly switching a pin on/off to control average power.
- **rsync** — File-copy tool that transfers only differences; used to push code to the Pi over SSH.
- **Slew rate limit** — Changing motor duty gradually instead of instantly, to avoid current spikes.
- **systemd unit / service** — Config file telling Linux to run a program at boot and restart it on failure.
- **Telemetry** — The stream of sensor/status data sent from the Pi to the browser.
- **TLS handshake** (`\x16\x03\x01`) — The first bytes of an HTTPS connection; seen as garbage 400 errors when a browser speaks HTTPS to an HTTP server.
- **venv** — Isolated Python environment so project packages don't touch the system Python.
- **WebSocket** — A persistent two-way connection over HTTP; carries commands down and telemetry up on one channel.